

Cyclistic Case Study: Analyze Phase

Shresth

2025-05-01

Introduction

This case study explores how annual members and casual riders use the Cyclistic bike-share system differently. The analysis is based on cleaned Q1 datasets from 2019 and 2020, prepared as part of the Google Data Analytics Capstone Project.

The business objective is to identify usage trends and behavioral differences that can inform a marketing strategy to convert more casual riders into annual members.

The following R analysis includes: - Descriptive statistics of ride patterns - Aggregated comparisons between user types - Visualizations of usage trends by day of the week and demographic segments

The ultimate goal is to derive actionable insights from user behavior that can support strategic decision-making.

Installing Packages

```
library(readxl)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##   filter, lag
```

```
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##   date, intersect, setdiff, union
```

```
library(janitor)
```

```
##  
## Attaching package: 'janitor'
```

```
## The following objects are masked from 'package:stats':  
##  
##   chisq.test, fisher.test
```

```
library(here)
```

```
## here() starts at E:/Github Projects/cyclist-case-study
```

```
library(scales)
```

Collect Data

We cleaned two files 'Divvy_trips_2019_Q1.xlsx' and 'Divvy_trips_2020_Q1.xlsx' and their cleaned data is saved in Cleaned_data directory, which is why we installed 'here' library to work with structured paths

```
df_2019 <- read_excel(here("Cleaned_data", "Divvy_Trips_2019_Q1_cleaned.xlsx"))  
df_2020 <- read_excel(here("Cleaned_data", "Divvy_Trips_2020_Q1_cleaned.xlsx"))
```

Preview of both dataset

```
head(df_2019)
```

```
## # A tibble: 6 × 15  
##   trip_id start_time      end_time      bikeid tripduration  
##   <dbl> <dtm>          <dtm>          <dbl>      <dbl>  
## 1 21742443 2019-01-01 00:04:37 2019-01-01 00:11:07    2167        390  
## 2 21742444 2019-01-01 00:08:13 2019-01-01 00:15:34    4386        441  
## 3 21742445 2019-01-01 00:13:23 2019-01-01 00:27:12    1524        829  
## 4 21742446 2019-01-01 00:13:45 2019-01-01 00:43:28     252       1783  
## 5 21742447 2019-01-01 00:14:52 2019-01-01 00:20:56    1170        364  
## 6 21742448 2019-01-01 00:15:33 2019-01-01 00:19:09    2437        216  
## # i 10 more variables: from_station_id <dbl>, from_station_name <chr>,  
## #   to_station_id <dbl>, to_station_name <chr>, usertype <chr>, gender <chr>,  
## #   birthyear <dbl>, ride_length <dtm>, day_of_week <chr>, age_group <chr>
```

```
head(df_2020)
```

```
## # A tibble: 6 × 15
##   ride_id      rideable_type started_at      ended_at
##   <chr>        <chr>      <dtm>      <dtm>
## 1 EACB19130B0CDA4A docked_bike 2020-01-21 20:06:59 2020-01-21 20:14:30
## 2 8FED874C809DC021 docked_bike 2020-01-30 14:22:39 2020-01-30 14:26:22
## 3 789F3C21E472CA96 docked_bike 2020-01-09 19:29:26 2020-01-09 19:32:17
## 4 C9A388DAC6ABF313 docked_bike 2020-01-06 16:17:07 2020-01-06 16:25:56
## 5 943BC3CBECCFD662 docked_bike 2020-01-30 08:37:16 2020-01-30 08:42:48
## 6 6D9C8A6938165C11 docked_bike 2020-01-10 12:33:05 2020-01-10 12:37:54
## # i 11 more variables: start_station_name <chr>, start_station_id <dbl>,
## #   end_station_name <chr>, end_station_id <dbl>, start_lat <dbl>,
## #   start_lng <dbl>, end_lat <dbl>, end_lng <dbl>, member_casual <chr>,
## #   ride_length <dtm>, day_of_week <chr>
```

Checking Column names for merging

```
colnames(df_2019)
```

```
## [1] "trip_id"      "start_time"    "end_time"
## [4] "bikeid"      "tripduration"  "from_station_id"
## [7] "from_station_name" "to_station_id" "to_station_name"
## [10] "usertype"     "gender"        "birthyear"
## [13] "ride_length"  "day_of_week"   "age_group"
```

```
colnames(df_2020)
```

```
## [1] "ride_id"      "rideable_type" "started_at"
## [4] "ended_at"     "start_station_name" "start_station_id"
## [7] "end_station_name" "end_station_id" "start_lat"
## [10] "start_lng"    "end_lat"       "end_lng"
## [13] "member_casual" "ride_length"   "day_of_week"
```

Combining into a single file

```
(df_2019<-rename(df_2019, ride_id = trip_id,
                 rideable_type=bikeid,
                 started_at = start_time,
                 ended_at = end_time,
                 start_station_name = from_station_name,
                 start_station_id = from_station_id,
                 end_station_name = to_station_name,
                 end_station_id = to_station_id,
                 member_casual = usertype))
```

```
## # A tibble: 365,053 × 15
##   ride_id started_at      ended_at      rideable_type tripduration
##   <dbl> <dtm>          <dtm>          <dbl>          <dbl>
## 1 21742443 2019-01-01 00:04:37 2019-01-01 00:11:07      2167      390
## 2 21742444 2019-01-01 00:08:13 2019-01-01 00:15:34      4386      441
## 3 21742445 2019-01-01 00:13:23 2019-01-01 00:27:12      1524      829
## 4 21742446 2019-01-01 00:13:45 2019-01-01 00:43:28       252     1783
## 5 21742447 2019-01-01 00:14:52 2019-01-01 00:20:56      1170      364
## 6 21742448 2019-01-01 00:15:33 2019-01-01 00:19:09      2437      216
## 7 21742449 2019-01-01 00:16:06 2019-01-01 00:19:03      2708      177
## 8 21742450 2019-01-01 00:18:41 2019-01-01 00:20:21      2796      100
## 9 21742451 2019-01-01 00:18:43 2019-01-01 00:47:30      6205     1727
## 10 21742452 2019-01-01 00:19:18 2019-01-01 00:24:54      3939      336
## # i 365,043 more rows
## # i 10 more variables: start_station_id <dbl>, start_station_name <chr>,
## #   end_station_id <dbl>, end_station_name <chr>, member_casual <chr>,
## #   gender <chr>, birthyear <dbl>, ride_length <dtm>, day_of_week <chr>,
## #   age_group <chr>
```

Inspecting the dataset

```
head(df_2019)
```

```
## # A tibble: 6 × 15
##   ride_id started_at      ended_at      rideable_type tripduration
##   <dbl> <dtm>          <dtm>          <dbl>          <dbl>
## 1 21742443 2019-01-01 00:04:37 2019-01-01 00:11:07      2167      390
## 2 21742444 2019-01-01 00:08:13 2019-01-01 00:15:34      4386      441
## 3 21742445 2019-01-01 00:13:23 2019-01-01 00:27:12      1524      829
## 4 21742446 2019-01-01 00:13:45 2019-01-01 00:43:28       252     1783
## 5 21742447 2019-01-01 00:14:52 2019-01-01 00:20:56      1170      364
## 6 21742448 2019-01-01 00:15:33 2019-01-01 00:19:09      2437      216
## # i 10 more variables: start_station_id <dbl>, start_station_name <chr>,
## #   end_station_id <dbl>, end_station_name <chr>, member_casual <chr>,
## #   gender <chr>, birthyear <dbl>, ride_length <dtm>, day_of_week <chr>,
## #   age_group <chr>
```

Mutating rows to same datatype for binding

'ride_id' in df_2019 is of double data type while in 2020 it is of character, which would throw error while binding, so we need to mutate the row to be of same data type. The same reason applies to 'rideable_type';

```
df_2019 <- mutate(df_2019, ride_id = as.character(ride_id),
                  rideable_type = as.character(rideable_type))
```

```
all_trips <- bind_rows(df_2019, df_2020)
head(all_trips)
```

```
## # A tibble: 6 × 19
##   ride_id started_at      ended_at      rideable_type tripduration
##   <chr>      <dtm>          <dtm>          <chr>              <dbl>
## 1 21742443 2019-01-01 00:04:37 2019-01-01 00:11:07 2167      390
## 2 21742444 2019-01-01 00:08:13 2019-01-01 00:15:34 4386      441
## 3 21742445 2019-01-01 00:13:23 2019-01-01 00:27:12 1524      829
## 4 21742446 2019-01-01 00:13:45 2019-01-01 00:43:28 252      1783
## 5 21742447 2019-01-01 00:14:52 2019-01-01 00:20:56 1170      364
## 6 21742448 2019-01-01 00:15:33 2019-01-01 00:19:09 2437      216
## # i 14 more variables: start_station_id <dbl>, start_station_name <chr>,
## #   end_station_id <dbl>, end_station_name <chr>, member_casual <chr>,
## #   gender <chr>, birthyear <dbl>, ride_length <dtm>, day_of_week <chr>,
## #   age_group <chr>, start_lat <dbl>, start_lng <dbl>, end_lat <dbl>,
## #   end_lng <dbl>
```

As it seems that certain rows are not required for the analysis, it is better to remove them.

```
all_trips <- all_trips %>%
  select(-c(start_lat, start_lng, end_lat, end_lng, birthyear, gender, tripduration, age_group))
head(all_trips)
```

```
## # A tibble: 6 × 11
##   ride_id started_at      ended_at      rideable_type start_station_id
##   <chr>      <dtm>          <dtm>          <chr>              <dbl>
## 1 217424... 2019-01-01 00:04:37 2019-01-01 00:11:07 2167      199
## 2 217424... 2019-01-01 00:08:13 2019-01-01 00:15:34 4386      44
## 3 217424... 2019-01-01 00:13:23 2019-01-01 00:27:12 1524      15
## 4 217424... 2019-01-01 00:13:45 2019-01-01 00:43:28 252      123
## 5 217424... 2019-01-01 00:14:52 2019-01-01 00:20:56 1170      173
## 6 217424... 2019-01-01 00:15:33 2019-01-01 00:19:09 2437      98
## # i 6 more variables: start_station_name <chr>, end_station_id <dbl>,
## #   end_station_name <chr>, member_casual <chr>, ride_length <dtm>,
## #   day_of_week <chr>
```

Data Cleaning and analysis

```
colnames(all_trips)
```

```
## [1] "ride_id"      "started_at"    "ended_at"
## [4] "rideable_type" "start_station_id" "start_station_name"
## [7] "end_station_id" "end_station_name" "member_casual"
## [10] "ride_length"   "day_of_week"
```

```
nrow(all_trips)
```

```
## [1] 791730
```

```
dim(all_trips)
```

```
## [1] 791730    11
```

```
head(all_trips)
```

```
## # A tibble: 6 × 11
##   ride_id started_at      ended_at      rideable_type start_station_id
##   <chr>   <dtm>         <dtm>         <chr>             <dbl>
## 1 217424... 2019-01-01 00:04:37 2019-01-01 00:11:07 2167             199
## 2 217424... 2019-01-01 00:08:13 2019-01-01 00:15:34 4386             44
## 3 217424... 2019-01-01 00:13:23 2019-01-01 00:27:12 1524             15
## 4 217424... 2019-01-01 00:13:45 2019-01-01 00:43:28 252             123
## 5 217424... 2019-01-01 00:14:52 2019-01-01 00:20:56 1170            173
## 6 217424... 2019-01-01 00:15:33 2019-01-01 00:19:09 2437             98
## # i 6 more variables: start_station_name <chr>, end_station_id <dbl>,
## #   end_station_name <chr>, member_casual <chr>, ride_length <dtm>,
## #   day_of_week <chr>
```

```
str(all_trips)
```

```
## tibble [791,730 × 11] (S3: tbl_df/tbl/data.frame)
##  $ ride_id      : chr [1:791730] "21742443" "21742444" "21742445" "21742446" ...
##  $ started_at   : POSIXct[1:791730], format: "2019-01-01 00:04:37" "2019-01-01 00:08:13"
##  ...
##  $ ended_at     : POSIXct[1:791730], format: "2019-01-01 00:11:07" "2019-01-01 00:15:34"
##  ...
##  $ rideable_type : chr [1:791730] "2167" "4386" "1524" "252" ...
##  $ start_station_id : num [1:791730] 199 44 15 123 173 98 98 211 150 268 ...
##  $ start_station_name: chr [1:791730] "Wabash Ave & Grand Ave" "State St & Randolph St" "Raci
ne Ave & 18th St" "California Ave & Milwaukee Ave" ...
##  $ end_station_id   : num [1:791730] 84 624 644 176 35 49 49 142 148 141 ...
##  $ end_station_name : chr [1:791730] "Milwaukee Ave & Grand Ave" "Dearborn St & Van Buren St
(*)" "Western Ave & Fillmore St (*)" "Clark St & Elm St" ...
##  $ member_casual    : chr [1:791730] "Subscriber" "Subscriber" "Subscriber" "Subscriber" ...
##  $ ride_length      : POSIXct[1:791730], format: "1899-12-31 00:06:30" "1899-12-31 00:07:21"
##  ...
##  $ day_of_week      : chr [1:791730] "Tuesday" "Tuesday" "Tuesday" "Tuesday" ...
```

```
summary(all_trips)
```

```
##      ride_id          started_at          ended_at
## Length:791730      Min.   :2019-01-01 00:04:37      Min.   :2019-01-01 00:11:07
## Class :character    1st Qu.:2019-02-28 16:59:45      1st Qu.:2019-02-28 17:10:03
## Mode  :character    Median :2020-01-07 12:25:02      Median :2020-01-07 12:40:28
##                      Mean   :2019-09-01 10:49:36      Mean   :2019-09-01 11:08:16
##                      3rd Qu.:2020-02-19 18:56:26      3rd Qu.:2020-02-19 19:14:15
##                      Max.   :2020-03-31 23:51:34      Max.   :2020-05-19 20:10:34
## rideable_type      start_station_id start_station_name end_station_id
## Length:791730      Min.   : 2.0      Length:791730      Min.   : 2.0
## Class :character    1st Qu.: 77.0      Class :character    1st Qu.: 77.0
## Mode  :character    Median :174.0      Mode  :character    Median :174.0
##                      Mean   :204.3              Mean   :204.3
##                      3rd Qu.:291.0              3rd Qu.:291.0
##                      Max.   :675.0              Max.   :675.0
## end_station_name    member_casual      ride_length
## Length:791730      Length:791730      Min.   :1899-12-31 00:00:01
## Class :character    Class :character    1st Qu.:1899-12-31 00:05:28
## Mode  :character    Mode  :character    Median :1899-12-31 00:08:57
##                      Mean   :1899-12-31 00:18:39
##                      3rd Qu.:1899-12-31 00:15:10
##                      Max.   :1900-04-17 15:30:24
## day_of_week
## Length:791730
## Class :character
## Mode  :character
##
##
##
```

The dataset contains 791730 rows and has the following dimensions: 791730 rows × 11 columns.

The member_casual of the aggregated dataset contains 4 kinds of values, but the user of cyclistic is divided into casual and members, so we need to make the data uniform by changing *Subscriber* and *Customer* to *member* and *casual* respectively or vice-versa. But we will go with 2020 labels.

```
unique(all_trips$member_casual)
```

```
## [1] "Subscriber" "Customer"   "member"     "casual"
```

Let's change the data to have uniform membership tag across the data set

```
all_trips <- all_trips %>% mutate(member_casual = recode(member_casual, "Subscriber" = "member",
"Customer" = "casual"))
unique(all_trips$member_casual)
```

```
## [1] "member" "casual"
```

Adding date details to the table

We already added `day_of_week` to the table while working on data with excel, but we might need other date details to aggregate ride data based for each day, month or year

```
all_trips$date <- as.Date(all_trips$started_at)
all_trips$month <- format(as.Date(all_trips$started_at), "%m")
all_trips$day <- format(as.Date(all_trips$started_at), "%d")
all_trips$year <- format(as.Date(all_trips$started_at), "%Y")
```

Final Data Integrity Check

Although the majority of data cleaning such as removing rides with negative or zero duration and unusually long trips was completed during the Excel preprocessing stage, we now perform a final integrity check within R.

In some cases, system-generated test rides (e.g., starting from "HQ QR" stations) or bad data entries may still exist in the merged dataset. These rides are not representative of real customer behavior and can bias the results.

The following step filters out: - Rides that started from "HQ QR" — often used by Divvy staff for testing or maintenance - Any remaining rides with negative ride lengths (in case of data merge inconsistencies)

We store the result in a new data frame for clean analysis going forward.

```
all_trips$ride_length <- difftime(all_trips$ended_at, all_trips$started_at, units = "mins")
all_trips$ride_length <- as.numeric(all_trips$ride_length)
```

```
all_trips <- all_trips[!(all_trips$start_station_name == "HQ QR" | all_trips$ride_length<0),]
```

Descriptive Analysis

Now that we got data cleaning out of the way, it's time to do some analysis. To understand how casual riders and annual members differ in their riding behavior, we'll start by comparing their ride durations. Below, we calculate the **mean**, **median**, **maximum**, and **minimum** ride lengths for each user type. This helps us see not only how long each group typically rides, but also how extreme the shortest and longest rides can get.

```
aggregate(all_trips$ride_length~all_trips$member_casual, FUN = mean)
```

```
##   all_trips$member_casual all_trips$ride_length
## 1                casual          83.89876
## 2                member          12.60708
```

```
aggregate(all_trips$ride_length~all_trips$member_casual,FUN = median)
```

```
##   all_trips$member_casual all_trips$ride_length
## 1                casual          23.216667
## 2                member           8.466667
```



```
aggregate(all_trips$ride_length~all_trips$member_casual,FUN = max)
```

```
##  all_trips$member_casual all_trips$ride_length
## 1          casual      156450.40
## 2          member      93793.52
```

```
aggregate(all_trips$ride_length~all_trips$member_casual,FUN = min)
```

```
##  all_trips$member_casual all_trips$ride_length
## 1          casual      0.03333333
## 2          member      0.01666667
```

Now let's find the average ride time based on the day of the week

```
aggregate(all_trips$ride_length~all_trips$member_casual + all_trips$day_of_week,FUN = mean)
```

```
##  all_trips$member_casual all_trips$day_of_week all_trips$ride_length
## 1          casual      Friday      94.43775
## 2          member      Friday      12.71021
## 3          casual      Monday      79.20084
## 4          member      Monday      12.38496
## 5          casual      Saturday     79.60064
## 6          member      Saturday     15.24707
## 7          casual      Sunday      84.35507
## 8          member      Sunday      14.97556
## 9          casual      Thursday     104.75618
## 10         member      Thursday     11.78682
## 11         casual      Tuesday      76.03006
## 12         member      Tuesday      11.87453
## 13         casual      Wednesday    70.85712
## 14         member      Wednesday    11.86640
```

It seems like the day of the week is not properly ordered, we need to fix that.

```
all_trips$day_of_week <- factor(all_trips$day_of_week,
                                levels = c("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",
                                "Friday", "Saturday"),
                                ordered = TRUE)
```

Now let's run the average time again

```
aggregate(all_trips$ride_length~all_trips$member_casual + all_trips$day_of_week,FUN = mean)
```

```
##      all_trips$member_casual all_trips$day_of_week all_trips$ride_length
## 1                casual      Sunday      84.35507
## 2                member      Sunday      14.97556
## 3                casual      Monday       79.20084
## 4                member      Monday      12.38496
## 5                casual      Tuesday      76.03006
## 6                member      Tuesday      11.87453
## 7                casual     Wednesday      70.85712
## 8                member     Wednesday      11.86640
## 9                casual     Thursday     104.75618
## 10               member     Thursday      11.78682
## 11               casual      Friday       94.43775
## 12               member      Friday      12.71021
## 13               casual     Saturday      79.60064
## 14               member     Saturday      15.24707
```

Since we need to work on visualization it is better to store the day of week as abbreviated terms in a new row.

```
all_trips$weekday <- wday(all_trips$started_at, label = TRUE, abbr = TRUE)
```

Let's check if `weekday` is a factor with the correct weekday order.

```
levels(all_trips$weekday)
```

```
## [1] "Sun" "Mon" "Tue" "Wed" "Thu" "Fri" "Sat"
```

Seems like it is properly ordered. Now it is time for *Visualization*

Visualisation

Now we'll use `ggplot2` to visualize the average ride duration and number of rides by rider type. But first, we'll store the summarized table in a variable to keep the plotting code clean and more readable.

```
summarized_table <- all_trips %>%
  group_by(member_casual, weekday) %>%
  summarise(
    number_of_rides = n(),
    average_duration = mean(ride_length, na.rm = TRUE)
  ) %>%
  arrange(member_casual, weekday)
```

```
## `summarise()` has grouped output by 'member_casual'. You can override using the
## `.groups` argument.
```

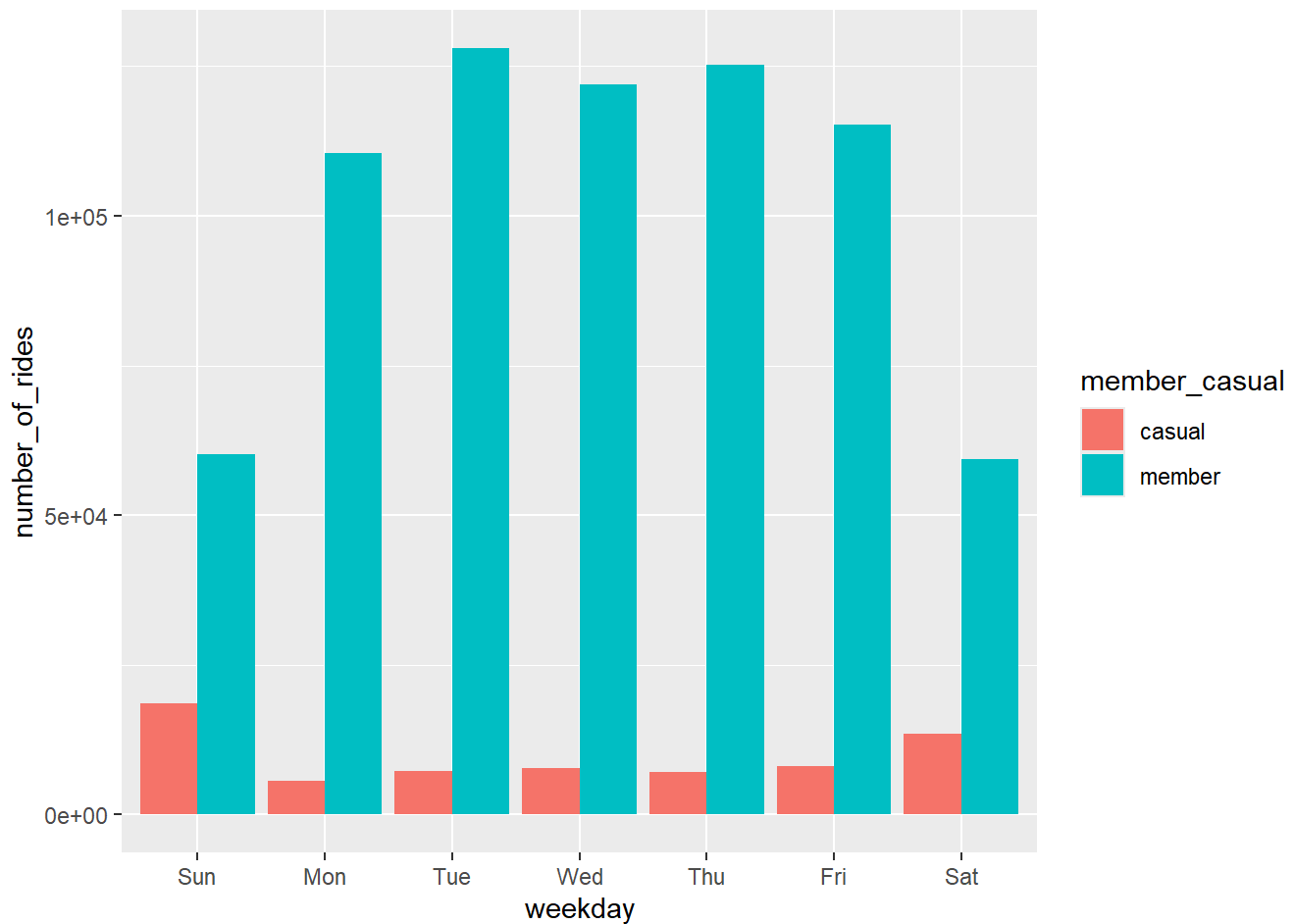
Checking the `summarized_table`

```
summarized_table
```

```
## # A tibble: 14 × 4
## # Groups:   member_casual [2]
##   member_casual weekday number_of_rides average_duration
##   <chr>         <ord>         <int>         <dbl>
## 1 casual      Sun           18652          84.4
## 2 casual      Mon           5591           79.2
## 3 casual      Tue           7311           76.0
## 4 casual      Wed           7689           70.9
## 5 casual      Thu           7145          105.
## 6 casual      Fri           8012           94.4
## 7 casual      Sat          13472           79.6
## 8 member      Sun          60195          15.0
## 9 member      Mon         110428          12.4
## 10 member     Tue         127972          11.9
## 11 member     Wed         121902          11.9
## 12 member     Thu         125228          11.8
## 13 member     Fri         115167          12.7
## 14 member     Sat          59409          15.2
```

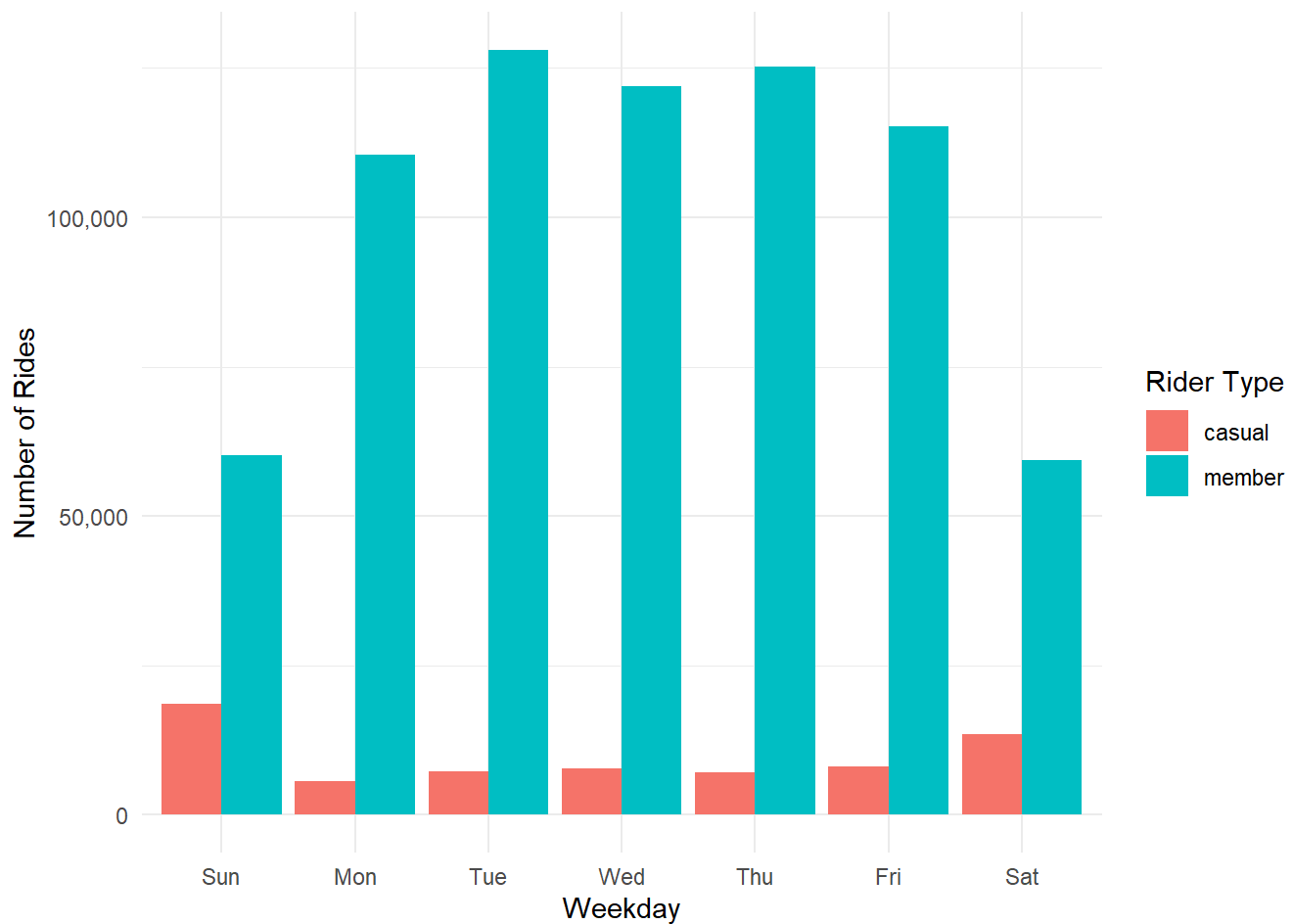
Now the data is ready for visualisation. Let's first create number of rides by rider type

```
ggplot(summarized_table, mapping = aes(weekday, number_of_rides, fill = member_casual)) + geom_bar(position = "dodge")
```



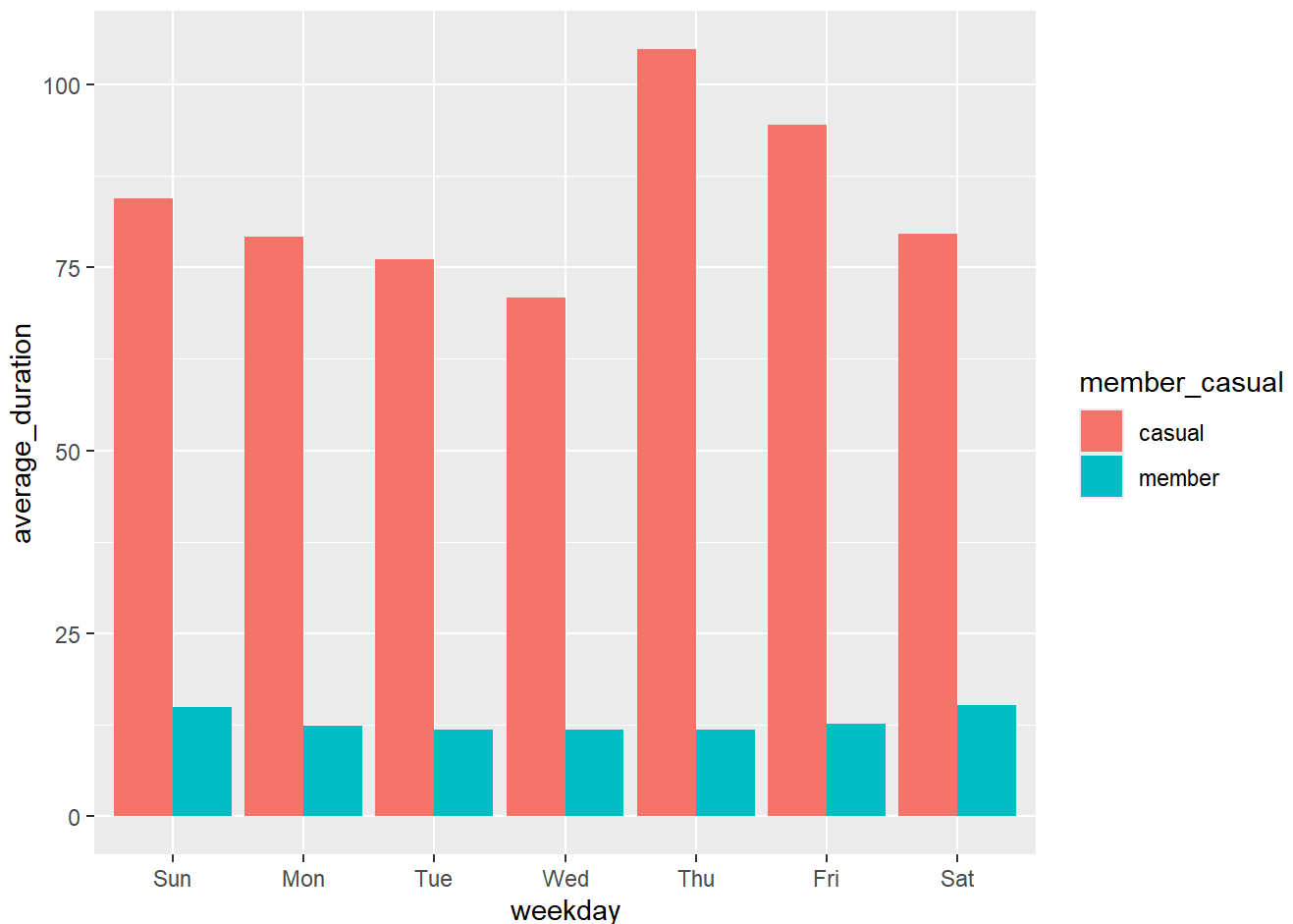
Well the visualisation works but the y-axis can be more human readable, let's change that.

```
ggplot(summarized_table, aes(x = weekday, y = number_of_rides, fill = member_casual)) +
  geom_col(position = "dodge") +
  scale_y_continuous(labels = comma) +
  labs(y = "Number of Rides", x = "Weekday", fill = "Rider Type") +
  theme_minimal()
```



Now let's see visualisation for average duration by biker type

```
ggplot(summarized_table, mapping = aes(x = weekday, y = average_duration, fill = member_casual))
+ geom_col(position = "dodge")
```



Conclusion

The analysis reveals clear differences in how casual riders and annual members use the Cyclistic bike-share system:

- **Ride Frequency:**

Annual members take significantly more rides during weekdays, suggesting that they use bikes for commuting or routine travel. In contrast, casual riders are more active on weekends, implying recreational use.

- **Ride Duration:**

Casual riders consistently have much longer average ride durations than members across all days of the week — especially on Thursdays and Fridays. This supports the idea that casual riders are exploring or riding for leisure rather than quick, practical trips.

Key Takeaways for the Marketing Team:

- To convert casual users into members, consider targeting **weekend riders** with incentives like “Weekend Membership Trials” or “Leisure Plans”.
- Casual riders might be more likely to convert if offered value based on **longer duration needs**, such as day passes that transition into discounted membership options.
- Messaging should be tailored around **exploration, flexibility, and savings for longer rides**, which casual users clearly favor.

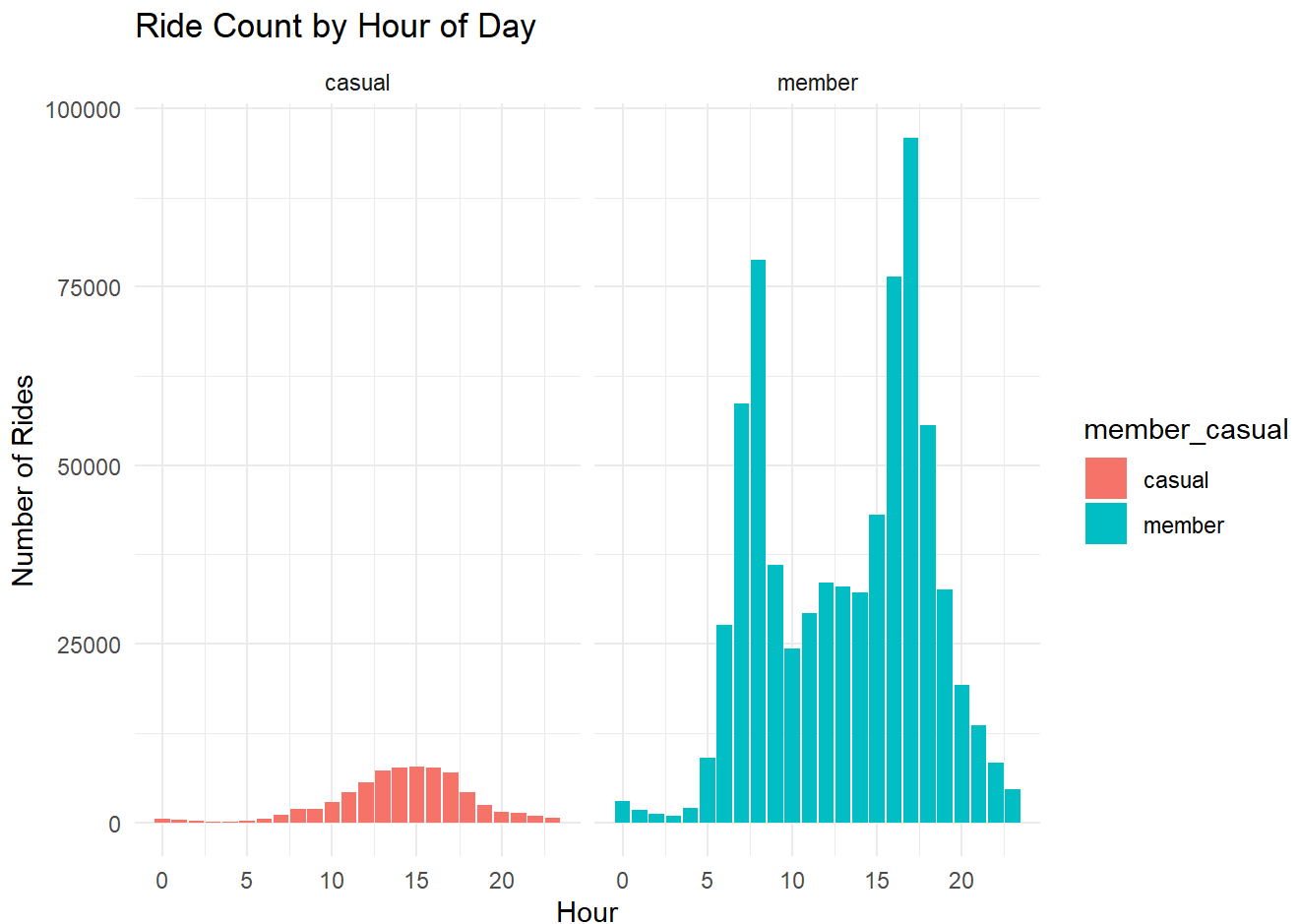
These insights will serve as the foundation for strategic recommendations in the next phase of the case study.

Bonus Visualizations

Beyond the core weekday analysis, we explored additional trends to better understand user behavior. These visualizations provide further insights that could support more targeted marketing strategies.

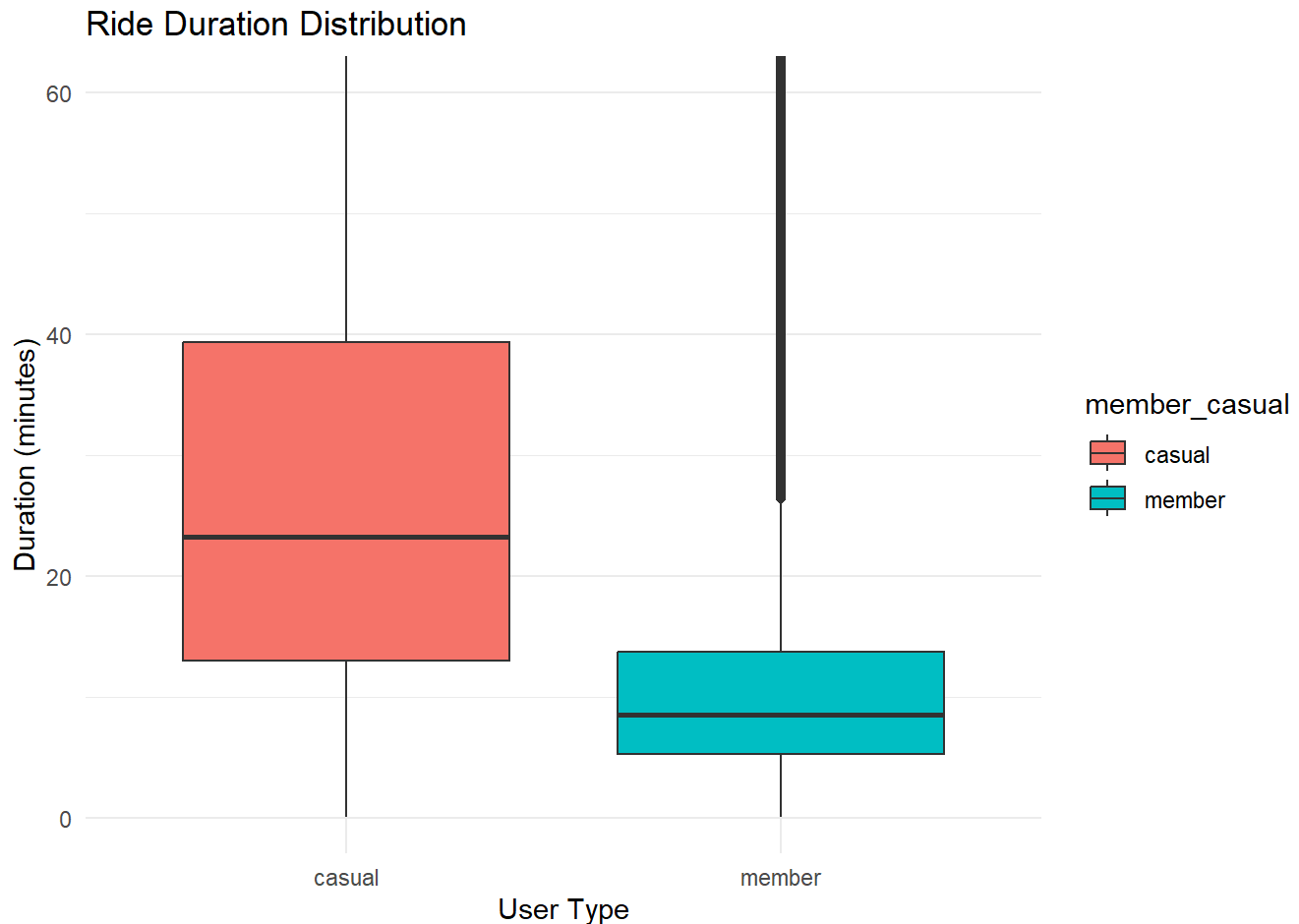
Ride count by hour of the day(faceted by user type)

```
all_trips %>%
  mutate(hour = hour(started_at)) %>%
  group_by(member_casual, hour) %>%
  summarise(rides = n(), .groups = "drop") %>%
  ggplot(aes(x = hour, y = rides, fill = member_casual)) +
  geom_col(position = "dodge") +
  facet_wrap(~member_casual) +
  labs(title = "Ride Count by Hour of Day", x = "Hour", y = "Number of Rides") +
  theme_minimal()
```



Ride duration distribution by user type (box plot)

```
ggplot(all_trips, aes(x = member_casual, y = ride_length, fill = member_casual)) +  
  geom_boxplot() +  
  coord_cartesian(ylim = c(0, 60)) + # Focus on rides < 60 min  
  labs(title = "Ride Duration Distribution", y = "Duration (minutes)", x = "User Type") +  
  theme_minimal()
```



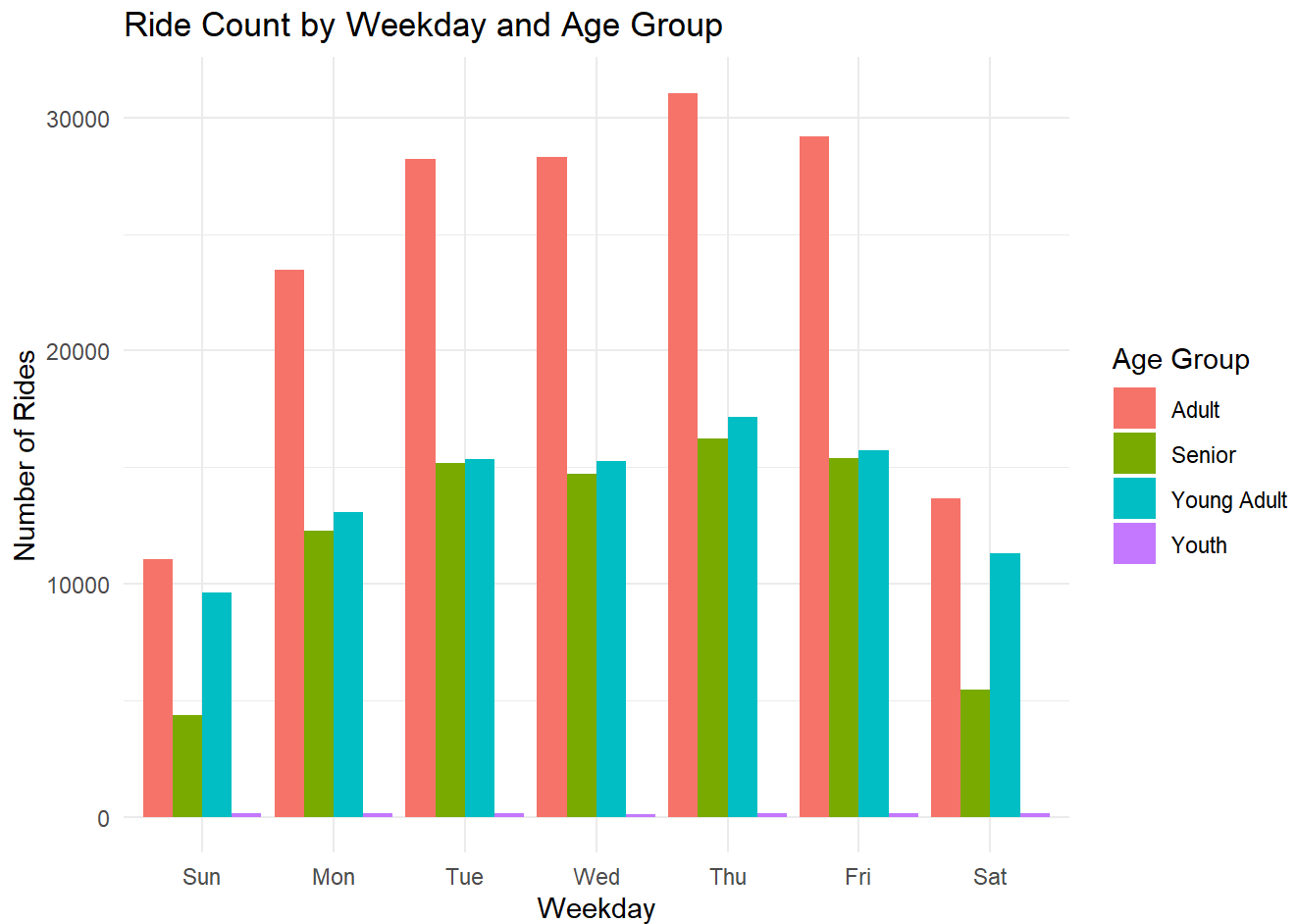
Rides by age group

```
df_2019$day_of_week <- factor(df_2019$day_of_week,  
                              levels = c("Sunday", "Monday", "Tuesday", "Wednesday", "Thursday",  
                                          "Friday", "Saturday"),  
                              ordered = TRUE)  
df_2019$weekday <- wday(df_2019$started_at, label = TRUE, abbr = TRUE)
```

```
age_weekday_summary <- df_2019 %>% filter(age_group!="Unknown") %>%  
  group_by(age_group, weekday) %>% summarise(rides=n(), .groups = "drop")
```

Grouped Bar chart

```
ggplot(age_weekday_summary, aes(x = weekday, y = rides, fill = age_group)) +
  geom_col(position = "dodge") +
  labs(title = "Ride Count by Weekday and Age Group",
       x = "Weekday", y = "Number of Rides", fill = "Age Group") +
  theme_minimal()
```



```
ggplot(age_weekday_summary, aes(x = weekday, y = rides, fill = weekday)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ age_group) +
  labs(title = "Ride Frequency by Weekday per Age Group",
       x = "Weekday", y = "Number of Rides") +
  theme_minimal()
```


Ride Frequency by Weekday per Age Group

